Guide de débogage sur ESP32

Du symptôme à la cause, avec les outils d'ESP-IDF

Michaël Roynard
michael.roynard@epita.fr

9 octobre 2026

Pourquoi déboguer un microcontrôleur est différent

Sur un poste de travail, un programme qui plante laisse un message, un code de retour et souvent un fichier d'image mémoire, puis le système continue de fonctionner. Sur un microcontrôleur, il n'y a pas de système d'exploitation hôte pour recueillir les morceaux : une faute mémoire, une pile qui déborde ou une tâche qui monopolise le processeur provoquent un **redémarrage de la carte**. Si rien n'a été prévu, le seul indice est une carte qui redémarre en boucle.

En effet, trois contraintes changent la manière de travailler. Premièrement, le code tourne sur une autre machine que l'outil qui l'observe : tout passe par un lien (UART, USB ou sonde JTAG). Deuxièmement, le système est temps réel : arrêter un processeur pour l'inspecter peut déclencher un chien de garde ou fausser une mesure. Enfin, la mémoire est comptée : une trace détaillée coûte de la flash, du tas et du temps.

C'est pourquoi l'industrie ne s'appuie pas sur un seul outil, mais sur une chaîne d'outils, du moins coûteux au plus intrusif.

La chaîne d'outils de l'industrie

Niveau	Outil	Ce qu'il apporte	Matériel requis
1	tests sur l'hôte	la logique pure (statistiques, analyse, formats) testée en millisecondes, sans carte	aucun
2	journaux (`ESP_LOGx`) et moniteur	ce que le programme dit de lui-même	UART
3	panique et backtrace décodée	où le programme est mort, et par quel chemin	UART
4	core dump	l'état complet au moment du crash, inspecté après coup avec GDB	UART (ou flash)
5	GDB stub	GDB attaché à la carte par l'UART, au crash ou à la demande	UART
6	JTAG (OpenOCD + GDB)	points d'arrêt matériels, pas à pas, sans modifier le firmware	sonde JTAG, intégrée à l'ESP32-S3
7	émulateur QEMU	GDB complet sans carte	aucun (ESP32 classique uniquement)

Le bon réflexe consiste à descendre le moins possible dans ce tableau. Un bug de logique se reproduit et se corrige sur l'hôte ; un crash se comprend le plus souvent avec la backtrace et le core dump ; le pas à pas sur la carte est réservé aux interactions fines avec le matériel ou l'ordonnanceur.

Ce que permet votre carte

	ESP32 classique (port UART seul)	ESP32-S3 (ports UART et USB natif)
journaux, backtrace, core dump	oui	oui
GDB stub (au crash, à la demande)	oui	oui
JTAG : points d'arrêt, pas à pas	seulement avec une sonde externe (par exemple ESP-Prog)	**oui, par le port USB natif** (JTAG intégré)
QEMU	oui	non (cible non émulée par ESP-IDF v6.0)

Avec une carte étudiante (ESP32 classique), vous disposez donc de tous les niveaux sauf le JTAG, et la combinaison core dump + GDB stub couvre l'essentiel des besoins.

Niveau 1 : figer le bug dans un test hôte

Tout ce qui ne dépend pas du matériel doit être testable sur le poste. Dans le TP2, les statistiques, le format JSONL, le validateur et l'analyse le sont déjà :

```
tools/run_host_tests.sh
```

Lorsque vous trouvez un bug de logique, écrivez d'abord le test qui le reproduit, vérifiez qu'il échoue, puis corrigez. Le test reste ensuite dans le projet et empêche le bug de revenir : c'est la pratique standard, sur cible comme ailleurs.

Niveau 2 : journaux et moniteur

```
idf.py -p "$PORT" flash monitor      # Ctrl+] pour quitter, Ctrl+T Ctrl+H pour l'aide
```

Les macros `ESP_LOGE`, `ESP_LOGW`, `ESP_LOGI`, `ESP_LOGD` et `ESP_LOGV` produisent des lignes horodatées et filtrables par niveau (`idf.py menuconfig`, *Component config* → *Log*). Journalisez les décisions et les codes d'erreur, jamais une clé, un nonce ou un texte clair. Un journal coûte : un `printf` dans une fenêtre chronométrée fausse la mesure.

Pour les codes PSA, quelques valeurs fréquentes (`psa/crypto_values.h`) :

Code	Nom	Cause typique
-133	`PSA_ERROR_NOT_PERMITTED`	usage ou algorithme absent de la politique de clé
-134	`PSA_ERROR_NOT_SUPPORTED`	algorithme non activé dans la configuration
-135	`PSA_ERROR_INVALID_ARGUMENT`	taille de clé, de nonce ou de buffer incohérente
-138	`PSA_ERROR_BUFFER_TOO_SMALL`	buffer de sortie trop petit (penser au tag AEAD)
-141	`PSA_ERROR_INSUFFICIENT_MEMORY`	souvent des clés jamais détruites qui s'accumulent
-149	`PSA_ERROR_INVALID_SIGNATURE`	tag ou signature refusés (attendu sur une entrée altérée)

Niveau 3 : lire une panique

Une faute fatale affiche un bloc `Guru Meditation Error`, l'état des registres et une **backtrace**. Le moniteur d'ESP-IDF la décode automatiquement en fonctions, fichiers et lignes, à condition que l'ELF de la compilation flashée soit dans le répertoire de build.

Message	Signification
`LoadProhibited` / `StoreProhibited`	lecture ou écriture à une adresse invalide, souvent un pointeur nul (`EXCVADDR` proche de 0)
`InstrFetchProhibited`	saut vers une adresse invalide (pointeur de fonction corrompu)
`IllegalInstruction`	code corrompu ou pile écrasée
`***ERROR*** A stack overflow in task`	la pile d'une tâche a débordé : réduire les buffers locaux ou agrandir la pile

Message	Signification
`Task watchdog got triggered`	une tâche n'a pas rendu la main : il manque un `vTaskDelay`
`assert failed`	une précondition vérifiée par le code n'est pas tenue

Prenons l'exemple d'une moyenne calculée sur zéro échantillon, observé sur un ESP32 classique (extrait de la sortie réelle du moniteur) :

```
Guru Meditation Error: Core  0 panic'ed (IntegerDivideByZero). Exception was unhandled.
--- 0x4000d096: __udivdi3 in ROM
--- 0x400d3e65: mean at main/main.c:11
--- 0x400d3e86: app_main at main/main.c:18
--- 0x400d36bd: main_task at .../components/freertos/app_startup.c:206
```

La première ligne décodée est l'endroit du crash (ici, la division 64 bits de la ROM), les suivantes le chemin qui y mène : `mean()`, appelée par `app_main()`. Sans le moniteur, la même traduction se fait à la main :

```
xtensa-esp32-elf-addr2line -pfiaC -e build/<projet>.elf 0x400d3e65
```

(`xtensa-esp32s3-elf-addr2line` pour un ESP32-S3.) Une backtrace marquée `CORRUPTED` signale le plus souvent une pile écrasée.

Niveau 4 : le core dump

Le core dump enregistre, au moment du crash, les registres et la pile de **toutes** les tâches. Il s'inspecte ensuite avec GDB, sans que la carte reste arrêtée. Dans `idf.py menuconfig`, *Component config → Core dump* :

- *Data destination* : **UART**. Le dump est imprimé dans le moniteur, qui le reconnaît et l'affiche décodé ;
- ou **Flash**. Le dump survit au redémarrage ; avec la table de partitions par défaut, ESP-IDF ajoute lui-même la partition nécessaire. Il se relit ensuite :

```
idf.py -p "$PORT" coredump-info      # résumé : tâche fautive, backtrace de chaque tâche
idf.py -p "$PORT" coredump-debug     # session GDB sur l'état figé
```

Sur l'exemple précédent, le core dump affiché par le moniteur donne directement la valeur fautive :

```
Crashed task handle: 0x3ffaf914, name: 'main'
#0  0x4000d099 in __udivdi3 ()
#1  0x400d5800 in mean (samples=0x3ffb4b20, count=0) at main/main.c:11
#2  0x400d5821 in app_main () at main/main.c:18
```

Dans `coredump-debug`, les commandes GDB usuelles s'appliquent : `bt`, `info threads`, `thread N`, `frame N`, `info locals`, `print variable`.

Ces options n'apparaissent dans `menuconfig` que si les composants `espcoredump` et `esp_gdbstub` font partie du projet. Les projets du TP2 les déclarent déjà ; un projet qui restreint la liste de ses composants (`set(COMPONENTS ...)` ou `MINIMAL_BUILD`) doit les y ajouter.

Niveau 5 : le GDB stub

Le GDB stub est un petit serveur GDB intégré au firmware, qui parle par l'UART.

- **Au crash** : *Component config → ESP System Settings → Panic handler behaviour = GDBStub on panic*. Au lieu de redémarrer, la carte affiche `Entering gdb stub now.` et attend GDB ; `idf.py monitor` le détecte et lance GDB, attaché à la carte figée.
- **À la demande** : *Component config → GDB Stub → GDBStub at runtime*. Dans `idf.py monitor`, `Ctrl+C` interrompt le programme et ouvre GDB attaché à la carte.

Ces deux modes ne demandent aucun matériel supplémentaire : c'est l'outil principal des cartes sans JTAG. Ils modifient en revanche le firmware : désactivez-les avant toute mesure de performance.

Niveau 6 : JTAG, OpenOCD et GDB

Le JTAG est le débogueur « complet » : points d'arrêt matériels, pas à pas, inspection de la mémoire, sans modifier le firmware. Sur l'ESP32-S3, le contrôleur JTAG est intégré au port **USB natif** ; sur l'ESP32 classique, il faut une sonde externe reliée aux broches JTAG.

Sous WSL2, attachez aussi le périphérique USB natif avec `usbipd`, puis installez une fois les règles udev d'OpenOCD (fichier `60-openocd.rules` fourni avec OpenOCD dans `~/.espressif/tools/openocd-esp32/`, à copier dans `/etc/udev/rules.d/`). Ensuite, dans deux terminaux ESP-IDF :

```
idf.py openocd                  # terminal 1 : serveur de débogage
idf.py gdb                      # terminal 2 : GDB connecté à OpenOCD
```

Dans GDB :

```
break measurement_task          # point d'arrêt
continue                        # reprendre jusqu'au point d'arrêt
next / step                     # pas à pas, sans ou avec entrée dans les appels
print valid_samples             # inspecter une variable
bt                              # pile d'appels
info threads                    # tâches FreeRTOS
```

Arrêter le processeur suspend aussi les chiens de garde et l'horloge de l'ordonnanceur : un temps mesuré pendant une session de débogage n'a aucune valeur.

Niveau 7 : QEMU, déboguer sans carte

ESP-IDF intègre l'émulateur QEMU d'Espressif pour l'ESP32 classique (pas pour l'ESP32-S3). Installation, une fois, puis exécution :

```
sudo apt install libgcrypt20 libglib2.0-0 libpixman-1-0 libsdl2-2.0-0 libslirp0
python3 "$IDF_PATH/tools/idf_tools.py" install qemu-xtensa
# le script d'activation d'EIM ne connaît pas encore QEMU : l'ajouter au PATH du terminal
export PATH="$(dirname "$(find "$IDF_TOOLS_PATH" -name qemu-system-xtensa -type f | head -n 1)"):$PATH"
idf.py set-target esp32
idf.py qemu monitor             # exécuter le firmware émulé
idf.py qemu gdb                 # même chose, avec GDB attaché dès le démarrage
```

QEMU est utile pour la logique et le pas à pas sans carte. Il n'émule pas fidèlement les temps ni tous les périphériques : **aucune mesure de performance ne vaut sous QEMU**.

Méthode

1. **Reproduire** : un bug qui ne se reproduit pas ne se corrige pas. Notez la commande, la configuration et la sortie complète.
2. **Lire le premier message d'erreur**, pas le dernier : la cascade qui suit est souvent une conséquence.
3. **Formuler une hypothèse unique** et la tester par le moyen le moins coûteux (test hôte, journal, puis core dump ou GDB).
4. **Figer le bug dans un test** quand c'est possible, puis corriger.
5. **Garder une trace** : symptôme, hypothèse, preuve, correctif. Dans le TP2, cette trace alimente la section « ce qui ne marche pas, et pourquoi » du rapport.

Pièges fréquents du TP2

Symptôme	Cause probable
correction `fail` avec `psa_status` -133	politique de clé incomplète (usage ou algorithme)
correction `fail` avec -149 sur un vecteur publié	mauvais ordre des arguments (clé, nonce, données associées) ou tag non concaténé au chiffré
mesures valides puis -141 au fil des itérations	une clé n'est pas détruite sur un chemin
`Task watchdog got triggered` pendant la mesure	`vTaskDelay(1)` absent ou placé dans la fenêtre
`stack overflow in task crypto-benchmark`	gros buffers locaux dans un cas asymétrique
la capture s'arrête sans erreur	le moniteur ou un autre programme occupe déjà le port série

Références

- Espressif, *Fatal Errors* : https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/fatal-errors.html

- Espressif, *Core Dump* : https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/core_dump.html
- Espressif, *JTAG Debugging* : https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/jtag-debugging/index.html
- Espressif, *QEMU Emulator* : https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/qemu.html
- Espressif, *IDF Monitor* : https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-guides/tools/idf-monitor.html